

Mathematical Foundations of Reinforcement Learning and Online Decision Making

Lecture 9 - Bandits for Test-Time Scaling

Sample-efficient verifier feedback with gap indices

Venktesh Viswanathan

Based on LiNGIFA (AISTATS 2021) and CASE (ICML'25),

29 September 2026

Overview

- 1 Test-time scaling and verifiers
- 2 Bandits recap: from arms to features
- 3 Gap-index exploration

How many of you know this?



The Hitchhiker's Guide to the Galaxy

Machine thinking: Deep Thought



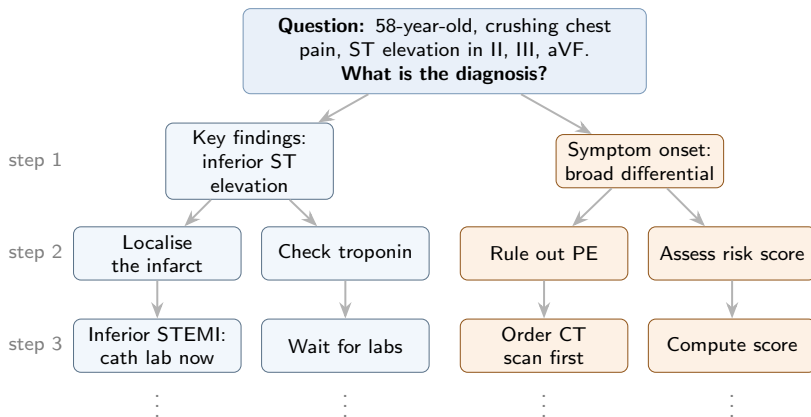
Deep Thought, a supercomputer, computes for 7.5 million years and comes up with the answer: **42**.

Modern LLMs do something similar, on a smaller scale: they **spend more compute at test time**, by generating many reasoning paths and searching among them.

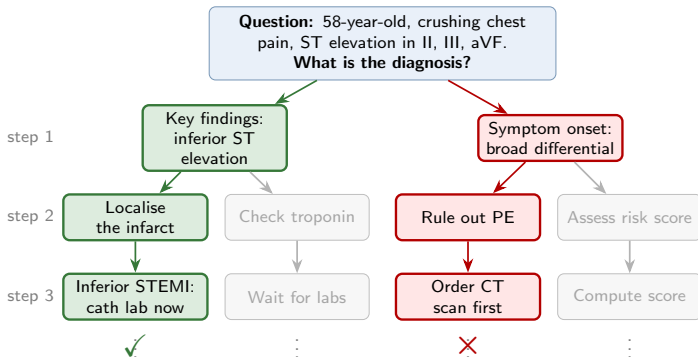
Test-time scaling: more thinking at inference time $\Rightarrow$ better answers, *if* we can tell good paths from bad ones.

How does Deep Thought compute?

So many possibilities! Every reasoning step branches.



How to select the correct reasoning path?



With branching factor b and depth D there are b^D paths: already $3^{10} = 59,049$ for 3 options over 10 steps.

Can we pick the right path without checking all of them?

Goal 1

We could keep expanding branches. What can be done more efficiently?

Goal

We could keep expanding branches. What can be done more efficiently?

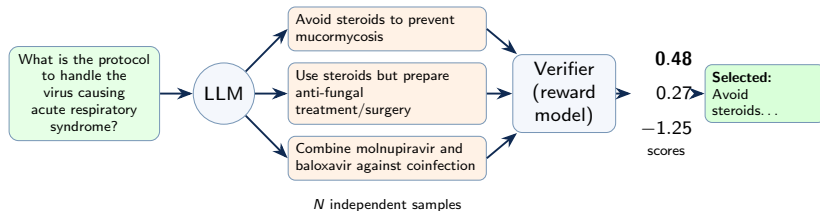
Group similar candidates

- Represent each solution as a vector: word embeddings, or word frequencies.

Keep this in mind: these vectors will become the **features** x_a of the bandit arms later.

Verification Oracle

A classifier that can identify not-so-fruitful solution paths?



This is **best-of- N** : sample N candidates, score each with a verifier, keep the highest-scoring one.

A simple binary classification task

- Frame verification as binary classification with labels
potentially correct = 1 and potentially wrong = 0. .

A simple binary classification task

- Frame verification as binary classification with labels **potentially correct** = 1 and **potentially wrong** = 0.
- The classifier gives intermediate verification of solution candidates.

A simple binary classification task

- Frame verification as binary classification with labels **potentially correct = 1** and **potentially wrong = 0**.
- The classifier gives intermediate verification of solution candidates.
- Any approach works: logistic regression, decision trees, ...
- **Training data:** mark solution paths that reach the right answer as 1, the others as 0.
- **How to choose?** If there are explicit rules to check against, decision trees are a good idea.

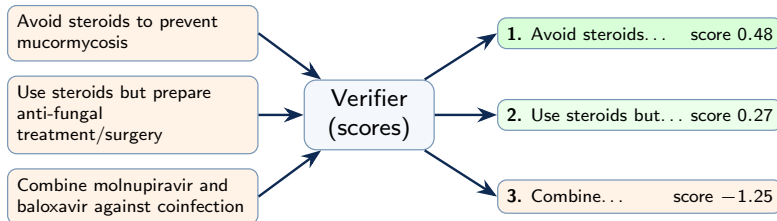
A simple binary classification task

- Frame verification as binary classification with labels **potentially correct = 1** and **potentially wrong = 0**.
- The classifier gives intermediate verification of solution candidates.
- Any approach works: logistic regression, decision trees, ...
- **Training data:** mark solution paths that reach the right answer as 1, the others as 0.
- **How to choose?** If there are explicit rules to check against, decision trees are a good idea.

In the LLM literature: an *outcome* reward model scores the final answer; a *process* reward model scores each intermediate step.

Alternative formulation: top- m instead of top-1

What if we want several top-ranked solutions, not just one? Frame the verifier as a **regression** (scoring) task and rank.

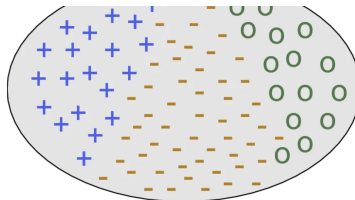


Top- m candidates feed majority voting, or get expanded further in a tree search.

The challenge we are trying to solve

Two strategies to assist Deep Thought:

- 1 cluster similar solution candidates;
- 2 classify or score each candidate as potentially correct or incorrect.



The **challenge**: navigating a **large search space** of candidates.

Verification is the bottleneck

Best-of- N with a verifier costs

$$\underbrace{N}_{\text{candidates}} \times \underbrace{n}_{\text{calls per candidate}} \times (\text{cost of one verifier call}).$$

- One call can be expensive: a large reward model, an LLM judge, running tests.

Verification is the bottleneck

Best-of- N with a verifier costs

$$\underbrace{N}_{\text{candidates}} \times \underbrace{n}_{\text{calls per candidate}} \times (\text{cost of one verifier call}).$$

- One call can be expensive: a large reward model, an LLM judge, running tests.
- Scores are **noisy**: a sampled LLM judge, or a step value estimated from a few rollouts, so one call per candidate is not enough ($n > 1$).

Verification is the bottleneck

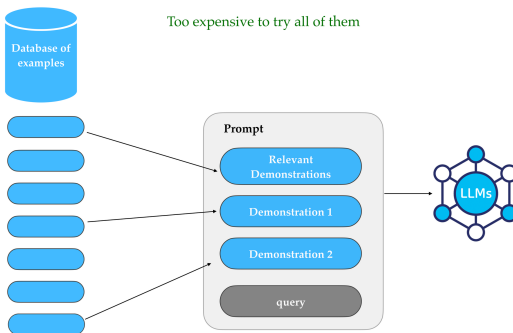
Best-of- N with a verifier costs

$$\underbrace{N}_{\text{candidates}} \times \underbrace{n}_{\text{calls per candidate}} \times (\text{cost of one verifier call}).$$

- One call can be expensive: a large reward model, an LLM judge, running tests.
- Scores are **noisy**: a sampled LLM judge, or a step value estimated from a few rollouts, so one call per candidate is not enough ($n > 1$).
- Example: $N = 64$ candidates, $n = 16$ rollouts each: 1,024 verifier calls for *one* question.

Which candidates deserve verifier calls? The ones whose ranking is still *uncertain*.

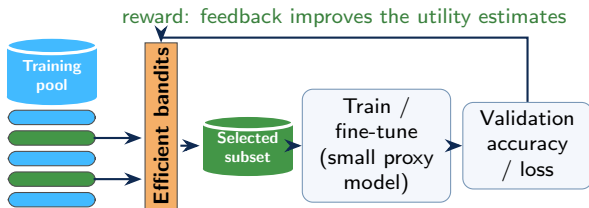
Motivation: choosing in-context exemplars



The answer depends on *which* k demonstrations are in the prompt; scoring one prompt costs LLM calls, and there are $\binom{n}{k}$ prompts.

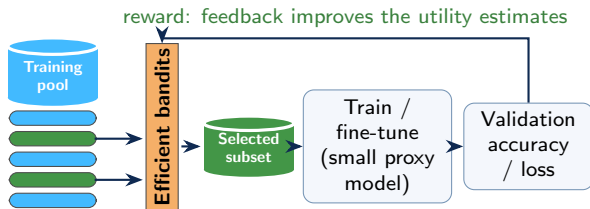
Exemplar subsets are arms; an LLM call is a pull.

Beyond prompts: data subset selection for training



- **Arm:** a data subset or source. **Pull:** train a (proxy) model and evaluate. **Reward:** validation score.

Beyond prompts: data subset selection for training



- **Arm:** a data subset or source. **Pull:** train a (proxy) model and evaluate. **Reward:** validation score.
- **Uses:** instruction-tuning data, data pruning, data mixtures.
- **Less can be more:** 1,000 curated examples sufficed to fine-tune LIMA; a small proxy model can set the data mixture for a large one (DoReMi).

Zhou et al., LIMA: Less Is More for Alignment, NeurIPS 2023. Xie et al., DoReMi: Optimizing Data Mixtures Speeds Up Language Model Pretraining, NeurIPS 2023.

Verification as a bandit problem

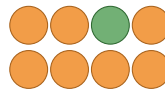
Bandit	Exemplar / Data subset selection	Test-time verification
arm a	subset of k exemplars	candidate reasoning path
pull	LLM call on a validation set	one verifier call
reward	accuracy / $-$ loss	verifier score
feature x_a	similarity features	embedding of the candidate
output	top- m subsets	top- m candidates

Same problem: find the top- m arms with **as few pulls as possible**. We develop the tools on exemplar selection, where they were introduced, and read them as verifier budgeting throughout.

Bandits - Recap



Candidates are arms



(exemplar subsets,
reasoning paths, ...)

Can I just exhaustively pull every arm?

Not unless you can see into the future



Exhaustive search means (number of arms) $\times$ (pulls per arm).
Arms can be combinatorial: 5-exemplar prompts from a pool of 100 give $\binom{100}{5} \approx 7.5 \times 10^7$ arms.

Bandits - Recap

arm	mean	time t					
		1	2	3	4	...	T
machine 1	5	5	5	5	5	...	5
machine 2	6	1	10	6	10	...	10
machine 3	2.5	2	3	3	2	...	2

Pink: the reward we actually observe (the arm we pulled). Grey: rewards we never see.

Balance exploration and exploitation

Bandits - Recap: feature design

Arms are not independent: they have **features**.

- Slot machines: slot size, age of the machine, ...

Bandits - Recap: feature design

Arms are not independent: they have **features**.

- Slot machines: slot size, age of the machine, ...
- Exemplar subsets: similarity of the exemplars to the task's questions.
- Reasoning paths: embedding of the path, number of steps, agreement with other candidates, ...

Bandits - Recap: feature design

Arms are not independent: they have **features**.

- Slot machines: slot size, age of the machine, ...
- Exemplar subsets: similarity of the exemplars to the task's questions.
- Reasoning paths: embedding of the path, number of steps, agreement with other candidates, ...

Similar features $\Rightarrow$ similar rewards. So one pull teaches us about many arms, just like the clusters of Idea 1.

Feature design

Linear stochastic bandits

Each arm a has a feature vector $x_a \in \mathbb{R}^d$, and an unknown $\alpha_* \in \mathbb{R}^d$:

$$\text{true reward: } \mu_a = x_a^\top \alpha_*,$$

$$\text{observed reward: } r_t = x_{a_t}^\top \alpha_* + \eta_t \quad (\eta_t \text{ noise}).$$

Estimate α_* by ridge regression on the pulls so far:

$$\hat{\alpha}_t = A_t^{-1} \sum_{s=1}^t x_{a_s} r_s, \quad A_t = \lambda I + \sum_{s=1}^t x_{a_s} x_{a_s}^\top,$$

and predict $\hat{\rho}_t(a) = x_a^\top \hat{\alpha}_t$.

Learn $\hat{\alpha}$ such that $\hat{\rho}_t(a) \approx \mu_a$ for all arms.

(A_t is the matrix V_t of Lecture 8.)

Pulling an arm = calling the LLM



$$\mu_a = x_a^\top \alpha_*$$
$$r_t = x_{a_t}^\top \alpha_* + \eta_t$$

Pull an arm, get an observable reward:

- exemplar selection: LLM accuracy or loss with that prompt;
- test-time scaling: the verifier's score of that candidate.

Every pull costs one (or more) expensive model calls.

A different goal: pure exploration

Lecture 8 (regret): maximize the reward collected *while* learning.

Here (identification): pulls earn nothing; they only cost verifier calls. We want the right answer at the end:

Top- m identification (fixed confidence)

Find a set $\hat{U}$ of m arms such that, with probability $\geq 1 - \delta$, no arm outside $\hat{U}$ beats an arm inside by more than ε , using as few pulls as possible.

- Cost measure: **sample complexity** (number of pulls), not regret.
- Quality of the output: **simple regret** (how far the chosen arms are from the best).

Loss modelling (approximation) for efficient selection

Exemplar pool $\mathcal{X} = \{x_1, \dots, x_n\}$, subset $S \subseteq \mathcal{X}$ of k exemplars, validation set $\mathcal{V} = \{u_1, \dots, u_{|\mathcal{V}|}\}$. Approximate the LLM loss $L(S, \mathcal{V})$ by

$$\sigma(\vec{\alpha}, S) = \frac{1}{|\mathcal{V}|} \sum_{j=1}^{|\mathcal{V}|} \sum_{i=1}^n \alpha_i \mathbf{1}[x_i \in S] E_{ij}.$$

- α_i : contribution of exemplar x_i , **low if it is an important exemplar**.
- E_{ij} : how similar exemplar x_i is to validation question u_j (next slide).

σ is linear in $\vec{\alpha}$: $\sigma(\vec{\alpha}, S) = \vec{\alpha}^\top x_S$ with $(x_S)_i = \mathbf{1}[x_i \in S] \frac{1}{|\mathcal{V}|} \sum_j E_{ij}$.
 A linear bandit with arm features x_S .

Similarity features

With any transformer-based encoder ϕ :

$$E_{ij} = \frac{\phi(x_i)^\top \phi(u_j)}{\|\phi(x_i)\| \|\phi(u_j)\|},$$

the cosine similarity between

- the i -th exemplar $x_i \in S$, and
- the j -th validation sample $u_j \in \mathcal{V}$.

An exemplar that resembles the questions we care about gets a large E_{ij} , so its weight α_i matters a lot for the predicted loss.

Good arms vs not-so-good arms

Let $\mathcal{S}$ be all k -subsets of $\mathcal{X}$; sample a collection $\mathcal{U} \subset \mathcal{S}$. At step t :

good arms



U_t

not-so-good arms



$V_t = \mathcal{U} \setminus U_t$

U_t : the l subsets with the lowest estimated validation loss.

V_t : everything else, used as a source of **negative** examples.

Efficient estimation of parameters

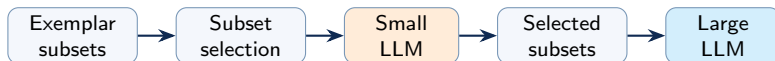
Update $\vec{\alpha}$ to reduce the approximation error on both groups:

$$\begin{aligned} \mathcal{L}(\vec{\alpha}; U_t, V_t) = & \sum_{S \in U_t} (L(S, \mathcal{V}) - \sigma(\vec{\alpha}, S))^2 \\ & + \sum_{S' \sim V_t} (L(S', \mathcal{V}) - \sigma(\vec{\alpha}, S'))^2. \end{aligned}$$

- First sum: fit the promising subsets accurately.
- Second sum: a few negative samples S' from the high-loss set V_t keep the model honest.

Computing $L(S, \mathcal{V})$ needs LLM calls: each one is an **arm pull**.

Tune and transfer



Select exemplars with a **small, cheap LLM**, then transfer them as in-context examples to a large LLM (e.g. GPT-3.5).

Same idea for verifiers: shortlist with a cheap verifier, confirm with the expensive one.

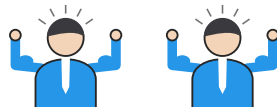
CASE - principled sampling of challengers

Current estimate of good arms



U_t

Challenger arms



$N_t \subset U_t^c$

CASE (Challenger Arm Sampling for Exemplar selection) compares the champions U_t (top- m by $\hat{\rho}_t$) only with a sampled shortlist N_t of challengers, not with *all* other arms.

Intuition - minimum-effort game



Each player picks an effort e_i ; the team's result is set by the **least** effort:

$$\text{payoff}_i = 2 \min_j e_j - e_i.$$

Extra effort above the weakest member is wasted.

The top- m set is only as trustworthy as its **weakest link**. Spend verifier calls there.

Van Huyck, Battalio & Beil (1990), *American Economic Review*.

How uncertain is a gap?

From Lecture 8: with probability $\geq 1 - \delta$, for every direction z ,

$$|z^\top (\hat{\alpha}_t - \alpha_*)| \leq \beta_t \|z\|_{A_t^{-1}}.$$

Take $z = x_a - x_b$, so $z^\top \alpha_* = \mu_a - \mu_b$ is the true **gap**:

$$|(\hat{\rho}_t(a) - \hat{\rho}_t(b)) - (\mu_a - \mu_b)| \leq \underbrace{\beta_t \|x_a - x_b\|_{A_t^{-1}}}_{=: W_t(a,b)}.$$

- $W_t(a, b)$: confidence width of the gap between a and b .
- It shrinks when we pull arms in the direction $x_a - x_b$, i.e. arms that tell a and b apart.

The gap index

$$B_t(a, b) = (\hat{\rho}_t(a) - \hat{\rho}_t(b)) + W_t(a, b)$$

- An **optimistic** upper bound on $\mu_a - \mu_b$: how much a could still beat b .
- $B_t(a, b) \leq 0 \Rightarrow$: a is not better than b .
- Large $B_t(a, b)$: either a looks better, or we are very unsure.

Same principle as UCB and OFUL: optimism in the face of uncertainty, now applied to *differences* between arms.

CASE - the weakest link b_t

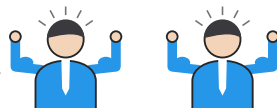
Current estimate of good arms



Challenger arms



U_t



$N_t \subset U_t^c$

$$b_t = \arg \max_{b \in U_t} \max_{a \in N_t} B_t(a, b)$$

The champion that *some* challenger is most likely to beat.

CASE - the strongest challenger s_t

Current estimate of good arms



Challenger arms



U_t

s_t

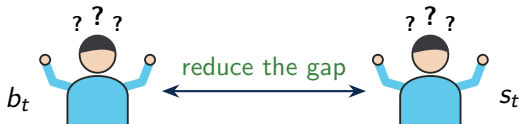


$N_t \subset U_t^c$

$$s_t = \arg \max_{c \in N_t} B_t(c, b_t)$$

The challenger most likely to beat the weakest champion: (b_t, s_t) is the most ambiguous comparison right now.

Convergence condition



$$B_t(s_t, b_t) = (\hat{\rho}_t(s_t) - \hat{\rho}_t(b_t)) + W_t(s_t, b_t)$$

Stopping rule

Stop when $B_t(s_t, b_t) \leq \varepsilon$: even the strongest challenger cannot beat the weakest champion by more than ε . Return U_t .

Until then, spend the next pull on the pair (b_t, s_t) .

Weakest link: where the uncertainty lives



$$B_t(s_t, b_t) = \underbrace{(\hat{\rho}_t(s_t) - \hat{\rho}_t(b_t))}_{\text{estimated gap}} + \underbrace{W_t(s_t, b_t)}_{\text{uncertainty}}$$

Often $W_t(s_t, b_t) \gg \hat{\rho}_t(s_t) - \hat{\rho}_t(b_t)$: the index is large not because s_t looks better, but because we cannot yet **tell the two apart**.

So pull arms that shrink $W_t(s_t, b_t)$: that is the most useful verifier call we can make.

Example

Challenger / U_t	Arm 2	Arm 4
Arm 1	0.3	0.1
Arm 3	0.2	0.5
Arm 5	0.4	0.2

$$K = 5, \quad m = 2$$

$$U_t = \{2, 4\}$$

$$N_t = \{1, 3, 5\}$$

Entry = gap index $B_t(\text{challenger}, U_t \text{ arm})$: how much the challenger could still beat that top arm.

For arm 2

Challenger / U_t	Arm 2	Arm 4
Arm 1	0.3	0.1
Arm 3	0.2	0.5
Arm 5	0.4	0.2

$$K = 5, \quad m = 2$$

$$U_t = \{2, 4\}$$

$$N_t = \{1, 3, 5\}$$

$$\max_{i \in \{1, 3, 5\}} B_t(i, 2) = \max(0.3, 0.2, 0.4) = 0.4$$

For arm 4

Challenger / U_t	Arm 2	Arm 4
Arm 1	0.3	0.1
Arm 3	0.2	0.5
Arm 5	0.4	0.2

$$K = 5, \quad m = 2$$

$$U_t = \{2, 4\}$$

$$N_t = \{1, 3, 5\}$$

$$\max_{i \in \{1, 3, 5\}} B_t(i, 4) = \max(0.1, 0.5, 0.2) = 0.5$$

Compute b_t

Challenger / U_t	Arm 2	Arm 4
Arm 1	0.3	0.1
Arm 3	0.2	0.5
Arm 5	0.4	0.2
max	0.4	0.5

$$K = 5, \quad m = 2$$

$$U_t = \{2, 4\}$$

$$N_t = \{1, 3, 5\}$$

$$b_t = \arg \max_{b \in U_t} \max_{a \in N_t} B_t(a, b) = ?$$

Compute b_t : answer

Challenger / U_t	Arm 2	Arm 4
Arm 1	0.3	0.1
Arm 3	0.2	0.5
Arm 5	0.4	0.2
max	0.4	0.5

$b_t = 4$ (weakest link)

$s_t = 3$ (its strongest challenger)

$B_t(s_t, b_t) = 0.5$

$0.5 > \varepsilon$: not converged. Pull to shrink $W_t(3, 4)$, re-estimate, possibly swap arms 3 and 4, and repeat until

$$B_t(s_t, b_t) = (\hat{\rho}_t(s_t) - \hat{\rho}_t(b_t)) + W_t(s_t, b_t) \approx 0.$$

CASE with a verifier

Top- m candidates with few verifier calls

Input: candidates with features x_a , m , shortlist size, ε , δ .

repeat

- ① $U_t \leftarrow$ top- m candidates by $\hat{\rho}_t$; sample challengers $N_t \subset U_t^c$.
- ② Swap if the best challenger looks better than the worst member of U_t .
- ③ $b_t = \arg \max_{b \in U_t} \max_{a \in N_t} B_t(a, b)$, $s_t = \arg \max_{c \in N_t} B_t(c, b_t)$.
- ④ **if** $B_t(s_t, b_t) \leq \varepsilon$: **return** U_t .
- ⑤ Call the verifier on the candidate that best shrinks $W_t(s_t, b_t)$; update A_t , $\hat{\alpha}_t$.

Verifier calls go only to the most ambiguous comparison.

Why this is sample efficient

- **Shared features:** one verifier call updates $\hat{\alpha}_t$, and hence the predicted score of *every* candidate.
- **Focused calls:** only the pair (b_t, s_t) gets verified; clear winners and clear losers are not re-checked.
- **Cheap bookkeeping:** gap indices only for $U_t \times N_t$, i.e. $m |N_t|$ pairs instead of $m(K - m)$.
- **Principled stop:** verification ends as soon as the ranking is certain enough $(B_t(s_t, b_t) \leq \varepsilon)$.

For exemplar selection, CASE reports up to $7\times$ fewer LLM calls than prior selection methods, without losing accuracy (ICML'25).

Takeaways

- Test-time scaling turns reasoning into **search**; the verifier is what makes search useful, and it is expensive and noisy.
- Candidates are **arms**, verifier calls are **pulls**, and embeddings give a **linear** bandit.
- The goal is **identification** (top- m with few pulls).

Takeaways

- The **gap index** $B_t(a, b)$ uses the Lecture 8 ellipsoid to say which comparison is still uncertain.
- CASE spends calls on the **weakest link** vs. its strongest challenger, over a small shortlist.

Open questions: offline policies, verifier feedback in tree search, cascades of cheap and expensive verifiers.

References

- K. Purohit, Venkatesh V, R. Devalla, K. M. Yerragorla, S. Bhattacharya, A. Anand. EXPLORA: Efficient exemplar subset selection for complex reasoning. *EMNLP*, 2024.
- K. Purohit, Venkatesh V, S. Bhattacharya, A. Anand. Sample efficient demonstration selection for in-context learning (CASE). *ICML*, 2025.
- L. Xu, J. Honda, M. Sugiyama. A fully adaptive algorithm for pure exploration in linear bandits (LinGapE). *AISTATS*, 2018.
- C. Réda, E. Kaufmann, A. Delahaye-Duriez. Top- m identification for linear bandits (LinGIFA). *AISTATS*, 2021.
- Y. Abbasi-Yadkori, D. Pál, Cs. Szepesvári. Improved algorithms for linear stochastic bandits. *NeurIPS*, 2011.
- K. Cobbe et al. Training verifiers to solve math word problems. *arXiv*, 2021.
- H. Lightman et al. Let's verify step by step. *ICLR*, 2024.
- X. Wang et al. Self-consistency improves chain of thought reasoning in language models. *ICLR*, 2023.
- C. Snell et al. Scaling LLM test-time compute optimally can be more effective than scaling model parameters. *ICLR*, 2025.
- J. B. Van Huyck, R. C. Battalio, R. O. Beil. Tacit coordination games, strategic uncertainty, and coordination failure. *American Economic Review*, 1990.